

Informe Técnico

Predicción de Satisfacción del Cliente en E-Commerce (Olist)

Evaluación Parcial N°2 — SCY1101

Programación para la Ciencia de Datos

Duoc UC — Mención Data Science

Autor: Guillermo Cerda

Framework: Kedro 1.4.0 + Scikit-learn + LightGBM + XGBoost + Optuna

Dataset: Brazilian E-Commerce Public Dataset (Olist) — Kaggle

Fecha: Mayo 2026

1. Resumen Ejecutivo

Este proyecto implementa un sistema de machine learning end-to-end para predecir la satisfacción de clientes del marketplace brasileño Olist, utilizando datos logísticos, de precio y de producto de aproximadamente 100,000 órdenes reales. El pipeline completo —desde la ingesta de 9 archivos CSV hasta la evaluación de 11 modelos entrenados— es ejecutable con el comando `kedro run` en ~452 segundos.

Objetivo: Clasificar órdenes como "satisfactorias" (`review_score > 2` estrellas) o "insatisfactorias" (1-2 estrellas), superando el umbral de rendimiento del 85% de accuracy.

Indicador	Valor
Dataset final	95,811 órdenes entregadas × 39 features
Balance de clases	87.2% satisfechos / 12.8% insatisfechos
Mejor accuracy (NB03, val-set)	89.54% — RandomForest (+4.54 pp sobre objetivo)
Mejor accuracy (NB04, Optuna end-to-end)	89.71% — XGBoost_Optuna (+4.71 pp sobre objetivo)
Mejor F1-Score (NB03)	0.9427 (RandomForest) — Recall positiva: ~98.7%
Mejor ROC-AUC	0.7756 (VotingEnsemble)
Modelos evaluados	11 (7 clasificadores base + 4 modelos Optuna)
Método de optimización	Optuna TPE: 100 trials LightGBM + 80 trials XGBoost
Clustering	KMeans k=2 óptimo (silhouette=0.5098)
PCA	2 componentes explican 86.8% de la varianza
Reproducibilidad	100% — Kedro 1.4.0, random_state=42 en todos los modelos

Nota sobre doble evaluación: Los modelos base (NB03) se evaluaron sobre PKLs entrenados con 32 features y threshold ajustado en val set → RandomForest lidera con 89.54%. Los modelos Optuna (NB04/save_best_models.py) se entrenaron end-to-end con 39 features → XGBoost_Optuna alcanza 89.71%.

Hallazgo clave: Las features de entrega (delivery_days, delay_days, delivery_ratio) dominan la predicción, confirmando que el tiempo de entrega es el driver principal de satisfacción en e-commerce. El techo de información del dataset es ~89.7%: superar este valor requeriría features de calidad del producto (descripciones, fotos) no disponibles en los datos logísticos.

2. Marco Metodológico

2.1 Justificación del enfoque: Clasificación binaria vs. Regresión

La primera decisión metodológica fue determinar si el problema se aborda como regresión (predecir el puntaje exacto 1-5) o como clasificación binaria (predecir satisfecho/insatisfecho). Se evaluaron dos modelos de regresión:

Modelo	R ²	RMSE	MAE
Ridge	0.1932	1.1578	0.8953
RandomForestReg	0.1975	1.1548	0.9003

Con $R^2 \approx 0.19$ y $RMSE \approx 1.15$ estrellas, la regresión no es viable: los datos logísticos explican solo el 19% de la varianza en el puntaje exacto. La diferencia entre 3, 4 y 5 estrellas depende de factores subjetivos (calidad del producto, experiencia post-venta) no capturados en los datos de entrega y precio. En cambio, la diferencia entre 1-2 estrellas (insatisfacción severa) y 3-5 estrellas sí está correlacionada con variables logísticas. Se define:

`is_satisfied = (review_score > 2)`

2.2 Selección de Algoritmos Supervisados

Se implementaron 7 clasificadores base más 4 modelos Optuna con justificación técnica diferenciada:

Modelo	Justificación técnica
LogisticRegression	Baseline lineal. Establece el piso de rendimiento. Rápido e interpretable.
DecisionTree	Modelo no paramétrico simple. Detecta reglas directas (ej: delay_days > X → insatisfecho).
RandomForest	Ensemble de bagging. Reduce varianza del DecisionTree promediando múltiples árboles.

GradientBoosting	Boosting clásico sklearn. Construye árboles secuenciales corrigiendo errores anteriores.
LightGBM	Boosting eficiente basado en histogramas. 10× más rápido que GradientBoosting para datasets >50K filas.
XGBoost	Boosting con regularización L1/L2. scale_pos_weight maneja el desbalance de clases nativamente.
VotingEnsemble (LGBM+XGB+GB)	Soft-voting con pesos [0.4, 0.4, 0.2]. Promedia probabilidades, reduce varianza individual.
CatBoost_Optuna	Manejo nativo de features categóricas. Resistente a overfitting por ordered boosting.
LightGBM_Optuna	LightGBM con hiperparámetros optimizados por Optuna TPE (100 trials).
XGBoost_Optuna	XGBoost con hiperparámetros optimizados por Optuna TPE (80 trials).
VotingEnsemble_Optuna	Soft-voting sobre los 3 mejores modelos Optuna.

2.3 Manejo del Desbalance de Clases

El dataset presenta desbalance severo: 87.2% satisfechos / 12.8% insatisfechos. Se evaluaron tres estrategias:

- **class_weight='balanced'**: Penaliza clase mayoritaria. Simple pero subóptimo para boosting.
- **ADASYN**: Muestras sintéticas adaptativas. Mayor tiempo de cómputo, sin mejora significativa.
- **SMOTE (sampling_strategy=0.25) [seleccionado]**: Sobremuestrea la minoría hasta que la clase 0 sea el 25% de la clase 1. Resultado: 66,842 filas train post-SMOTE (80%/20%). SMOTE se aplica solo sobre X_train (no sobre el val set), evitando contaminación del threshold tuning.

Principio de no contaminación: SMOTE se aplica exclusivamente al conjunto de entrenamiento. El test set (19,163 filas) mantiene la distribución original (87.2%/12.8%) para evaluar el rendimiento real en condiciones de producción.

2.4 Threshold Tuning

Con datasets desbalanceados, el threshold por defecto (0.5) es subóptimo. Se implementa búsqueda exhaustiva del umbral de decisión en [0.20, 0.81] con paso 0.01, seleccionando el valor que maximiza accuracy en el **validation set** (sin ver y_test, eliminando data leakage). Los thresholds óptimos encontrados varían entre 0.20 (LogisticRegression/DecisionTree) y 0.65 (LightGBM_Optuna), demostrando que el umbral por defecto habría penalizado el rendimiento entre 0.1 y 0.4 pp.

2.5 Feature Engineering: 7 Features de Interacción

Basado en investigación de patrones de e-commerce 2025-2026, se agregaron 7 features de interacción que capturan relaciones no lineales entre variables:

Feature	Fórmula	Razón técnica
price_x_delivery	$\text{price_per_item} \times \text{delivery_ratio.clip}(5)$	Productos caros que llegan tarde generan mayor insatisfacción
freight_x_delay	$\text{freight_ratio} \times \text{delay_days.clip}(0)$	Flete alto combinado con atraso amplifica la insatisfacción
delivery_efficiency	$\text{days_early} / \text{estimated_days.clip}(1)$	Eficiencia relativa de la entrega vs. promesa
purchase_quarter	$(\text{purchase_month} - 1) // 3 + 1$	Efectos estacionales (Q4 mayor volumen y atrasos)
is_fast_delivery	$\text{delivery_days} \leq 5$	Flag de entrega express (predictor binario fuerte)
is_very_late	$\text{delay_days} > 7$	Flag de atraso severo (>1 semana de demora)
total_cost_proxy	$\text{total_price} + \text{total_freight}$	Costo total percibido por el cliente

3. Análisis Experimental

3.1 Dataset: Brazilian E-Commerce Public Dataset (Olist)

Dataset público disponible en Kaggle, publicado por Olist (marketplace brasileño). Contiene datos reales de órdenes entre 2016 y 2018. Se utilizaron 8 de los 9 archivos disponibles:

Archivo CSV	Registros originales	Descripción
olist_orders_dataset.csv	99,441	Órdenes con timestamps, estado y claves foráneas
olist_customers_dataset.csv	99,441	Datos de clientes y estado geográfico
olist_order_items_dataset.csv	112,650	Items por orden: precio unitario y flete
olist_order_payments_dataset.csv	103,886	Métodos de pago, cuotas y montos
olist_order_reviews_dataset.csv	100,000	Reseñas y puntajes (variable target)
olist_products_dataset.csv	32,951	Dimensiones y peso de productos
olist_sellers_dataset.csv	3,095	Estado del vendedor
product_category_name_translation.csv	71	Traducción de categorías PT→EN

3.2 Pipeline Kedro de Procesamiento

El pipeline Kedro implementa 5 etapas secuenciales con 12 nodos totales:

Pipeline	Nodos	Entrada	Salida principal	Tiempo
----------	-------	---------	------------------	--------

data_ingestion	2	9 CSV raw	dataset_merged.csv (99,441 × 30)	~15s
data_cleaning	3	dataset_merged	dataset_clean.csv (95,811 × 28)	~12s
data_transform	3	dataset_clean	dataset_features.csv (95,811 × 34)	~18s
data_validation	2	dataset_features	dataset_validated.csv (95,811 × 32)	~8s
modeling	2	dataset_validated	métricas + comparacion_optimizacion.csv	~399s
TOTAL	12	9 CSV raw	5 datasets intermedios + métricas	~452s

Transformaciones de limpieza aplicadas:

- Filtro de órdenes entregadas: 99,441 → 95,811 filas (-3.6%)
- Winsorización IQR×1.5 en total_price y total_freight (outliers extremos)
- Imputación con mediana para valores nulos en columnas numéricas críticas
- LabelEncoder para 5 variables categóricas (order_status, payment_type, customer_state, seller_state, product_category)
- StandardScaler en columnas numéricas no binarias (media=0, std=1)

3.3 Hallazgos del Análisis Exploratorio

El EDA (notebook 01) reveló los siguientes patrones determinantes para el modelado:

Hallazgo	Detalle	Impacto en modelado
Distribución target	5★=59.3%, 4★=19.7%, 3★=8.3%, 2★=3%, 1★=9.7%	Binarización >2 natural: 87.2%/12.8%
Predictor principal	delivery_days y delay_days: separación clara entre clases en boxplots	Features de entrega son los más importantes
Correlaciones bajas	Correlación máxima entre features numéricas ≈ 0.30	Anticipa techo de información ~89.7%
Outliers en precios	total_price y total_freight con colas muy largas	Requiere Winsorización IQR
Nulos esperados	22 columnas con nulos (timestamps de entrega)	Eliminados con filtro de órdenes entregadas
Concentración geográfica	São Paulo (SP) ≈ 40% de órdenes	Variable customer_state relevante

3.4 Configuración Experimental

Parámetro	Valor	Justificación
random_state / SEED	42	Reproducibilidad en todos los modelos y particiones
test_size	0.20 (19,163 filas)	Estándar 80/20 para datasets >50K filas

stratify	y	Mantiene proporción 87.2%/12.8% en train y test
SMOTE sampling_strategy	0.25	Clase 0 = 25% de clase 1 → balance 80%/20%
Threshold search	[0.20, 0.81], paso=0.01	Búsqueda exhaustiva en rango significativo
CV folds	5 (StratifiedKFold)	Mantiene proporción de clases en cada fold
Optuna trials LightGBM	100	Convergencia confirmada: curva plateó a los ~80 trials
Optuna trials XGBoost	80	Convergencia más rápida por regularización L1/L2
Optuna sampler	TPESampler(seed=42)	Búsqueda Bayesiana reproducible

4. Resultados y Comparación de Modelos

4.1 Tabla Comparativa: 11 Clasificadores

#	Modelo	Accuracy	F1-Score	ROC-AUC	Threshold	Tipo
1	RandomForest	0.8954	0.9427	0.7586	0.45	Base
2	VotingEnsemble_Optuna	0.8938	0.9413	0.7735	0.52	Ensemble Optuna
3	GradientBoosting	0.8930	0.9408	0.7743	0.63	Base
4	DecisionTree	0.8928	0.9410	0.7282	0.21	Base
5	XGBoost_Optuna	0.8927	0.9406	0.7717	0.32	Optuna TPE 80 trials
6	VotingEnsemble	0.8926	0.9405	0.7756	0.51	Ensemble base
7	LightGBM_Optuna	0.8922	0.9402	0.7714	0.65	Optuna TPE 100 trials
8	LogisticRegression	0.8918	0.9403	0.7535	0.20	Baseline lineal
9	XGBoost	0.8917	0.9399	0.7737	0.35	Base
10	LightGBM	0.8912	0.9396	0.7707	0.62	Base
11	CatBoost_Optuna	0.8905	0.9392	0.7683	0.36	Optuna

Mejor modelo evaluado (NB03, 11 modelos): **RandomForest con 89.54% accuracy** — supera el objetivo de 85% en +4.54 pp. Los modelos Optuna entrenados end-to-end con 39 features (NB04/save_best_models.py) alcanzan hasta 89.71% (XGBoost_Optuna). El gap entre el mejor (89.54%) y el peor (CatBoost_Optuna, 89.05%) en la evaluación de modelos guardados es de 0.49 pp, evidenciando un techo de información de ~89.7% con los features logísticos disponibles.

4.2 Análisis Detallado de Métricas

Métrica	RandomForest	VotingEnsemble	Interpretación
Accuracy	89.54%	89.26%	% predicciones correctas en test real (19,163 filas)
F1-Score	0.9427	0.9405	Balance precisión-recall, útil con desbalance de clases
ROC-AUC	0.7586	0.7756	Capacidad de discriminar: VotingEnsemble gana en AUC
Precisión (clase positiva)	90.26%	90.97%	De los predichos satisfechos, los que realmente lo son
Recall (clase positiva)	98.65%	97.34%	De los satisfechos reales, los que el modelo captura
Threshold óptimo	0.45	0.51	Umbral hallado por búsqueda en val set [0.20, 0.81]

Trade-off clave: RandomForest maximiza Accuracy y Recall (98.65%). VotingEnsemble maximiza ROC-AUC (0.7756). Para el problema de negocio, maximizar Recall es prioritario: no detectar un cliente insatisfecho (Falso Negativo) es más costoso que clasificar erróneamente un satisfecho. Los modelos Optuna entrenados end-to-end con 39 features alcanzan 89.71% superando a todos los modelos base.

4.3 Análisis de la Matriz de Confusión (RandomForest, threshold=0.45)

	Predicho: Insatisfecho (0)	Predicho: Satisfecho (1)
Real: Insatisfecho (0)	TN = 667 (Recall negativa: 27.2%)	FP = 1,784 (Error principal del modelo)
Real: Satisfecho (1)	FN = 224 (Clientes satisfechos mal clasificados)	TP = 16,488 (Recall positiva: 98.7%)

El modelo captura el **98.7%** de los clientes satisfechos pero solo identifica correctamente el **27.2%** de los insatisfechos como tales (FP=1,784). Estos 1,784 falsos positivos representan clientes insatisfechos predichos erróneamente como satisfechos — el principal riesgo operativo del modelo. Este comportamiento es esperado dado el severo desbalance del dataset (87.2%/12.8%): el modelo prioriza no perder clientes satisfechos (alto Recall positivo) a costa de clasificar mal una fracción de los insatisfechos.

4.4 Validación Cruzada (cv=5, StratifiedKfold)

Modelo	CV Accuracy (media ± std)	Test Accuracy	Gap CV→Test	Interpretación
GradientBoosting	0.8952 ± 0.0012	0.8930	-0.0022	Estable en CV; threshold conservador en val set
LightGBM	0.8952 ± 0.0011	0.8912	-0.0040	CV más estable: menor std entre folds

RandomForest	0.8950 ± 0.0014	0.8954	+0.0004	Mejor test resultado; muy bajo gap CV→Test
VotingEnsemble	0.8937 ± 0.0015	0.8926	-0.0011	Ensemble estable, leve caída con val-threshold
XGBoost	0.8693 ± 0.0018	0.8917	+0.0224	Gap por ausencia de SMOTE en CV
LogisticRegression	0.8229 ± 0.0026	0.8918	+0.0689	Modelo lineal, beneficia más del SMOTE
DecisionTree	0.7982 ± 0.0101	0.8928	+0.0946	Alta varianza entre folds

GradientBoosting y LightGBM empatan en CV Mean (0.8952) con la menor std, siendo los más estables. RandomForest tiene el menor gap CV→Test (+0.0004). Los gaps mayores en XGBoost y LogisticRegression se explican por la ausencia de SMOTE dentro del CV — el CV evalúa el modelo PKL con class_weight implícito, no el pipeline completo con balanceo externo.

5. Modelos No Supervisados

5.1 KMeans Clustering

Se aplicó KMeans con k=2 a 10, evaluando inercia y coeficiente de silhouette para determinar el número óptimo de clusters:

k	Inercia	Silhouette	Observación
2	23,938,480	0.5098	Óptimo (mayor silhouette)
3	16,424,977	0.4018	
4	14,847,940	0.3072	
5	13,504,230	0.2641	
6	12,350,292	0.2420	
7	11,373,329	0.2497	
8	10,935,770	0.2191	

Resultado: k=2 es el número óptimo (silhouette=0.5098). La caída abrupta al pasar de k=2 (0.51) a k=3 (0.40) confirma una estructura binaria natural en los datos. Los dos clusters se alinean con satisfechos (bajo delay_days, alta delivery_efficiency) e insatisfechos (alto delay_days, alta delivery_ratio). El clustering no supervisado corrobora de forma independiente la definición del target binario.

5.2 Análisis de Componentes Principales (PCA)

Componente	Varianza explicada	Varianza acumulada
PC1	0.7908 (79.1%)	0.7908 (79.1%)
PC2	0.0769 (7.7%)	0.8677 (86.8%)
PC3	0.0427 (4.3%)	0.9104 (91.0%)
PC4	0.0352 (3.5%)	0.9456 (94.6%)
PC5	0.0157 (1.6%)	0.9613 (96.1%)
PC6	0.0084 (0.8%)	0.9697 (97.0%)
PC7	0.0060 (0.6%)	0.9757 (97.6%)
PC8	0.0049 (0.5%)	0.9807 (98.1%)

Resultado: Solo 2 componentes explican el 86.8% de la varianza (PC1=79.1%, PC2=7.7%). La alta concentración en PC1 evidencia que las 39 features están altamente correlacionadas entre sí: `delivery_days`, `estimated_days` y `delay_days` capturan esencialmente la misma dimensión subyacente (tiempo de entrega). Con 32 componentes se alcanza el 100% de la varianza. **Implicación práctica:** para un sistema productivo, reducir a 15-20 features de mayor importancia reduciría la latencia de inferencia sin pérdida significativa en accuracy.

5.3 Importancia de Features (LightGBM)

#	Feature	Importancia	Interpretación
1	<code>delivery_days</code>	1,438	Días reales de entrega — predictor dominante
2	<code>delay_days</code>	1,380	Días de atraso vs. estimado por el vendedor
3	<code>max_installments</code>	1,220	Cuotas máximas de pago — refleja expectativa del cliente
4	<code>delivery_ratio</code>	1,186	<code>delivery_days</code> / <code>estimated_days</code> (eficiencia relativa)
5	<code>freight_per_kg</code>	989	Costo de flete por peso del producto
6	<code>volume_cm3</code>	984	Volumen del producto (relacionado con flete)
7	<code>product_category</code>	940	Categoría del producto (variación por tipo)
8	<code>product_height_cm</code>	902	Dimensión física del producto
9	<code>customer_state</code>	878	Estado geográfico del cliente (variación regional)
10	<code>price_per_item</code>	856	Precio unitario del producto

6. Optimización de Hiperparámetros

6.1 GridSearchCV — LightGBM

Justificación: LightGBM es el modelo más rápido del conjunto (basado en histogramas, no árboles exactos), lo que permite explorar más combinaciones en el mismo tiempo que XGBoost o GradientBoosting.

Espacio de búsqueda: $2 \times 2 \times 2 \times 2 = 16$ combinaciones $\times$ 3 folds = 48 entrenamientos.

Hiperparámetro	Valores evaluados	Mejor valor	Razón
n_estimators	[300, 500]	500	Más árboles = mejor aproximación del gradiente
learning_rate	[0.03, 0.05]	0.05	Convergencia más rápida con dataset grande
max_depth	[5, 7]	7	Mayor profundidad captura interacciones más complejas
num_leaves	[31, 63]	63	$63 > 31$: más hojas por árbol = mayor capacidad

Resultados: CV accuracy = 0.8417 | Test accuracy = **89.04%** | Tiempo: ~26s

6.2 RandomizedSearchCV — XGBoost

Justificación: XGBoost tiene un espacio de hiperparámetros mayor (6 parámetros vs 4 de LightGBM). Búsqueda exhaustiva requeriría miles de combinaciones; RandomizedSearch explora 15 puntos aleatorios del espacio continuo de manera eficiente.

Espacio: 6 hiperparámetros = 432 combinaciones posibles | 15 iteraciones $\times$ 3 folds = 45 entrenamientos.

Hiperparámetro	Rango evaluado	Mejor valor	Razón
n_estimators	[300, 500, 700]	700	Más árboles con learning_rate bajo = mejor generalización
learning_rate	[0.01, 0.03, 0.05]	0.03	Tasa más baja compensa los 700 estimadores
max_depth	[4, 5, 6, 7]	7	Árboles más profundos con subsample como regularización
subsample	[0.6, 0.7, 0.8, 1.0]	0.8	Estocasticidad que reduce overfitting
colsample_bytree	[0.6, 0.7, 0.8, 1.0]	0.7	30% de features omitidas por árbol

Resultados: CV accuracy = 0.8409 | Test accuracy = **89.15%** | Tiempo: ~32s

6.3 Optuna — Búsqueda Bayesiana TPE

Justificación: GridSearch y RandomSearch exploran el espacio de forma ciega. Optuna usa el algoritmo TPE (Tree-structured Parzen Estimator) que aprende qué regiones del espacio son prometedoras y dirige la búsqueda hacia ellas, logrando mejores resultados con menos evaluaciones totales.

Ventajas sobre métodos clásicos:

- Espacio continuo: learning_rate=0.028 es inaccesible para GridSearch con valores discretos.
- Búsqueda adaptiva: cada trial informa al siguiente vía modelo probabilístico.
- 9 hiperparámetros simultáneos vs. 4-6 de los métodos clásicos (incluye reg_alpha, reg_lambda).
- Reproducible: TPESampler(seed=42) garantiza la misma secuencia de trials.

Mejores parámetros Optuna LightGBM (100 trials):

n_estimators=338, learning_rate=0.0189, max_depth=5, num_leaves=106, min_child_samples=68, subsample=0.956, colsample_bytree=0.913

Mejores parámetros Optuna XGBoost (80 trials):

n_estimators=520, learning_rate=0.0355, max_depth=8, subsample=0.669, colsample_bytree=0.934, reg_alpha=2.38e-8, reg_lambda=2.328

6.4 Tabla Comparativa de Métodos

Método	Modelo	Evaluaciones	Test Accuracy	Tiempo	Ventaja
GridSearchCV	LightGBM	16 combos × 3 folds = 48	89.04%	~26s	Exhaustivo, reproducible
RandomizedSearchCV	XGBoost	15 iter × 3 folds = 45	89.15%	~32s	Eficiente para espacios grandes
Optuna TPE	LightGBM	100 trials	89.60%	~79s	Espacio continuo, adaptivo
Optuna TPE	XGBoost	80 trials	89.71%	~78s	Regularización L1/L2 óptima

6.5 Análisis del Impacto

Comparación	Accuracy base	Accuracy óptimo	Mejora
GridSearch vs. modelo base LightGBM (NB03)	89.12% (LightGBM base)	89.04% (GridSearch)	-0.08 pp (GridSearch usa 32 features sin SMOTE en CV)
Optuna vs. GridSearch	89.04%	89.60%	+0.56 pp
Optuna end-to-end vs. mejor base (NB03)	89.54% (RandomForest base)	89.71% (XGBoost_Optuna)	+0.17 pp

Conclusión sobre optimización: Optuna supera a GridSearch en +0.56 pp utilizando espacios continuos y búsqueda adaptiva. El mejor resultado global es XGBoost_Optuna con 89.71% (entrenamiento end-to-end con 39 features). El dataset presenta un techo de información de ~89.7%: para superar este techo sería necesario incorporar features de calidad del producto (texto de la descripción, cantidad de fotos, rating del vendedor).

7. Conclusiones y Recomendaciones

7.1 Conclusiones Técnicas

- **Objetivo superado:** Se alcanzó 89.71% accuracy con XGBoost_Optuna (NB04, end-to-end con 39 features), superando el objetivo de 85% en +4.71 pp. En la evaluación de los 11 modelos guardados (NB03, 32 features), RandomForest lidera con 89.54% (+4.54 pp).
- **Clasificación binaria correcta:** $R^2=0.19$ confirma que la regresión sobre el puntaje exacto no es viable. La clasificación binaria (review_score > 2) es el enfoque correcto para los datos logísticos disponibles.
- **Techo de información ~89.7%:** Con 39 features logísticos, la mejora marginal de adicionar más modelos o trials es <0.05 pp. Para mejoras sustanciales se requieren features de calidad del producto.
- **Optuna > GridSearch/RandomSearch:** La búsqueda Bayesiana TPE supera a los métodos clásicos en +0.56 pp (89.60% vs. 89.04%) explorando espacios continuos con 9 hiperparámetros simultáneos.
- **Driver principal: tiempo de entrega.** Las features delivery_days y delay_days dominan el ranking de importancia. Las empresas de e-commerce deben priorizar velocidad y puntualidad de entrega para maximizar satisfacción.
- **Clustering corrobora target:** KMeans k=2 (silhouette=0.51) confirma de forma no supervisada la estructura binaria del problema.
- **Pipeline Kedro reproducible:** El comando `kedro run` reproduce todo el flujo end-to-end en ~452 segundos con `random_state=42`.

7.2 Dificultades Encontradas y Soluciones

- **Desbalance 87.2%/12.8%:** Requirió experimentación con SMOTE, ADASYN y class_weight. SMOTE con `sampling_strategy=0.25` resultó superior en validación.
- **Data leakage en threshold tuning:** El threshold de decisión debe buscarse en un validation set separado, no en el test set. Se implementó split en tres conjuntos (train 64% / val 16% / test 20%) para evitar que la búsqueda del umbral inflara artificialmente las métricas.
- **Feature mismatch en evaluación (32 vs. 39 features):** Los modelos Optuna se entrenaron con 39 features (incluyendo 7 de interacción) mientras los modelos base usaban 32. Se implementó enrutamiento automático basado en `n_features_in_`.
- **CatBoost retorna `n_features_in_=0`:** Comportamiento por diseño. Se implementó lógica especial: `if (n >= 39 or n == 0):` usar `X_test_opt`.
- **Threshold tuning omitido en CV:** La validación cruzada de los modelos PKL no incluye SMOTE ni threshold tuning interno, lo que genera gaps CV→Test más altos para XGBoost y LogisticRegression. Se documentó como limitación conocida del pipeline.

7.3 Recomendaciones para Trabajo Futuro

- **Features de producto:** Incorporar longitud de la descripción, cantidad de fotos y rating histórico del vendedor. Literatura indica mejora potencial de +2-3 pp en accuracy.

- **Modelo de producción optimizado:** Reducir a los top 15-20 features por importancia reduce latencia de inferencia sin pérdida significativa (<0.1 pp accuracy).
 - **Monitoreo de drift:** Implementar detección de data drift (ej. Evidently AI) para alertar cuando el perfil de órdenes cambia significativamente (Black Friday, Navidad).
 - **Stacking ensemble:** Meta-learner LogisticRegression sobre predicciones de LightGBM + XGBoost + CatBoost. Primeros experimentos sugieren mejora de $+0.01-0.02$ pp.
 - **Análisis por segmento:** Modelos especializados por categoría de producto (electrónica, muebles) podrían capturar patrones específicos que el modelo global ignora.
-

8. Referencias

1. Olist Store. (2018). Brazilian E-Commerce Public Dataset by Olist. Kaggle.
 2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD '16 Proceedings. <https://doi.org/10.1145/2939672.2939785>
 3. Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.
 4. Akiba, T., et al. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. KDD '19 Proceedings. <https://doi.org/10.1145/3292500.3330701>
 5. Chawla, N. V., et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321–357.
 6. Prokhorenkova, L., et al. (2018). CatBoost: unbiased boosting with categorical features. NeurIPS 2018.
 7. Kedro Development Team. (2024). Kedro Documentation v1.4. QuantumBlack, McKinsey & Company. <https://kedro.org>
 8. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
 9. Bergstra, J., & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. Journal of Machine Learning Research, 13, 281–305.
-